INTRODUCTION TO THE INTERNET OF THINGS · LAB 5

Lab 5 — Automation: Threshold-Triggered Relay & Buzzer

Time: ~22 minutes in the Telecom & Data-Centre Edition (~30 minutes on your own) **You'll learn:** how to make a device act on its own using a simple if-then rule — this is edge computing, made visible.

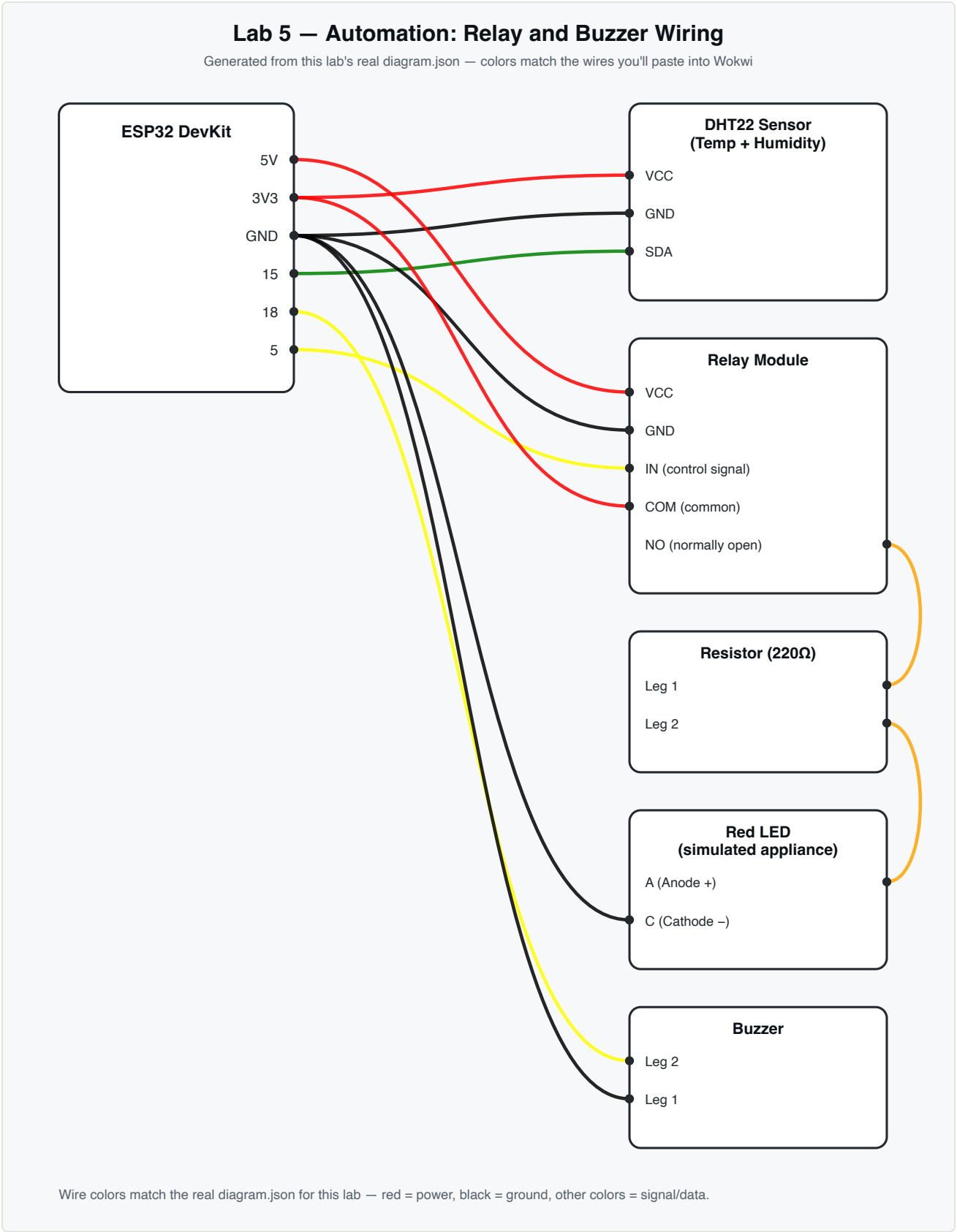
> **Telecom & Data-Centre Edition**
>
> **The story:** The shelter at a tower site gets too hot, so the relay switches on **extra cooling** and the buzzer sounds an alarm, without waiting for anyone.
>
> **Your team:** Teams of three. Rotate the roles from the last lab: **builder** (keyboard), **analyst** (reads the guide, checks the output), **decision owner** (answers the manager task).
>
> **Manager task:** What should happen automatically at the site, and what needs a person to confirm first?
>
> **Extension (fast teams):** Stop the cooling flickering on and off with **hysteresis**: on above 28 °C, off only below 26 °C. In `sketch.ino`, change one line. Replace
>
> `cpp if (data.temperature > TEMPERATURE_THRESHOLD_C) {`
>
> with
>
> `cpp if (data.temperature > TEMPERATURE_THRESHOLD_C || (alarmIsOn && data.temperature >= 26.0)) {`
>
> Read it aloud: *switch on above 28; once on, stay on until it drops below 26.*
>
> **Business-case canvas:** Fill in box 4 (the decision and the action).

What you're building

A DHT22 sensor, a **relay module** (an electrically-controlled switch that can turn a much bigger circuit on or off), and a **buzzer**. The code watches the temperature and, the moment it crosses a limit, switches the relay and buzzer on by itself — no button press, no cloud, no person involved.

The LED wired through the relay's `COM` / `NO` pins represents a real appliance (a fan, a warning light, an irrigation valve) being switched by the relay — this is different from the buzzer and DHT22, which the ESP32 controls directly.

Wiring diagram

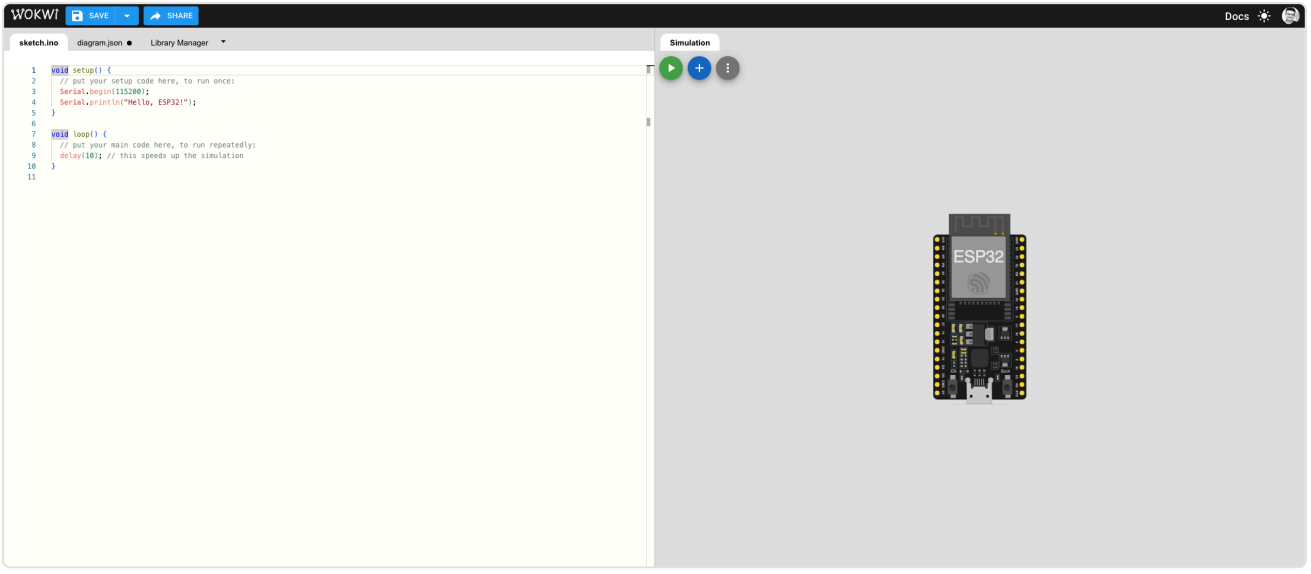


ESP32 Pin	Wire Color	Connects To
`5V`	red	Relay Module (VCC)
`3V3`	red	DHT22 Sensor (VCC), Relay Module (COM)
`GND`	black	DHT22 Sensor (GND), Relay Module (GND), simulated-appliance LED (Cathode), Buzzer
`15`	green	DHT22 Sensor (data pin)
`5`	yellow	Relay Module (IN — the control signal from the ESP32)
`18`	yellow	Buzzer

The relay's `NO` ("normally open") and `COM` pins switch a *separate* little circuit (resistor → LED) — that's the "simulated appliance," wired through the relay rather than straight to a GPIO pin, exactly like a real fan or pump would be.

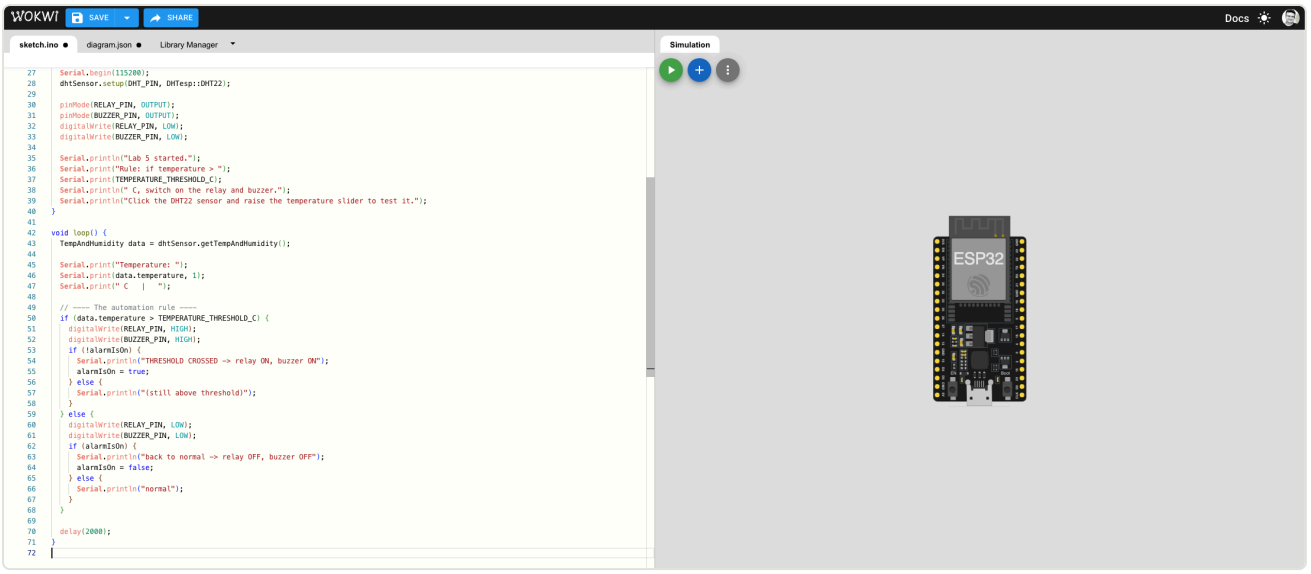
Step-by-step

1. Open a new ESP32 project



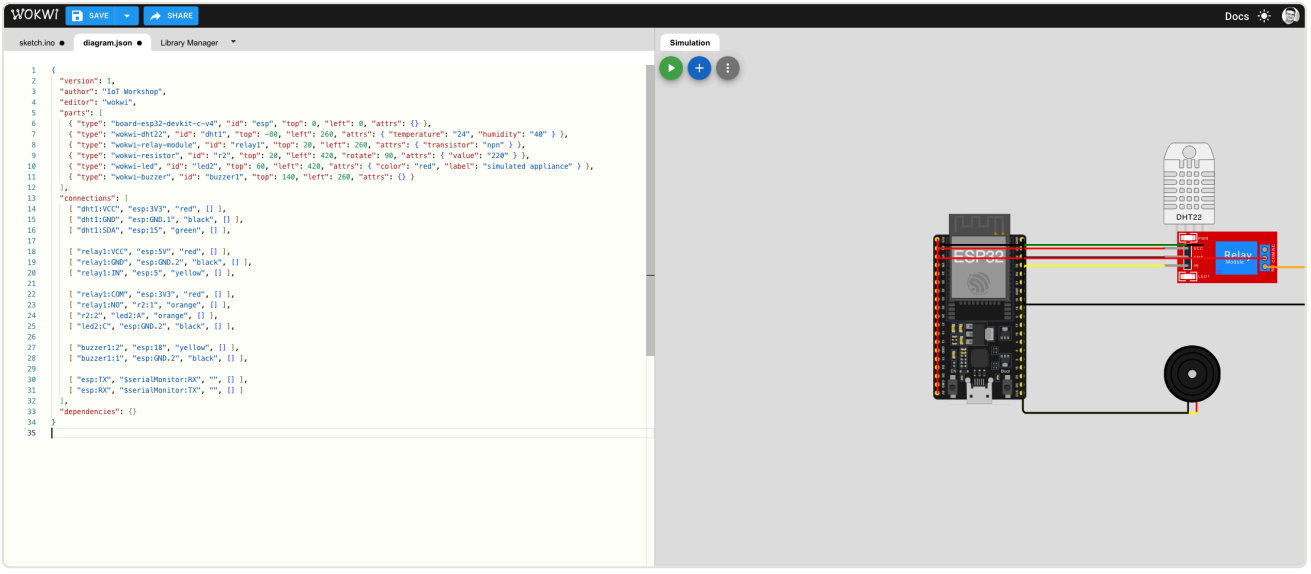
2. Paste the code

Copy `sketch.ino` into Wokwi's **sketch.ino** tab.



3. Paste the circuit

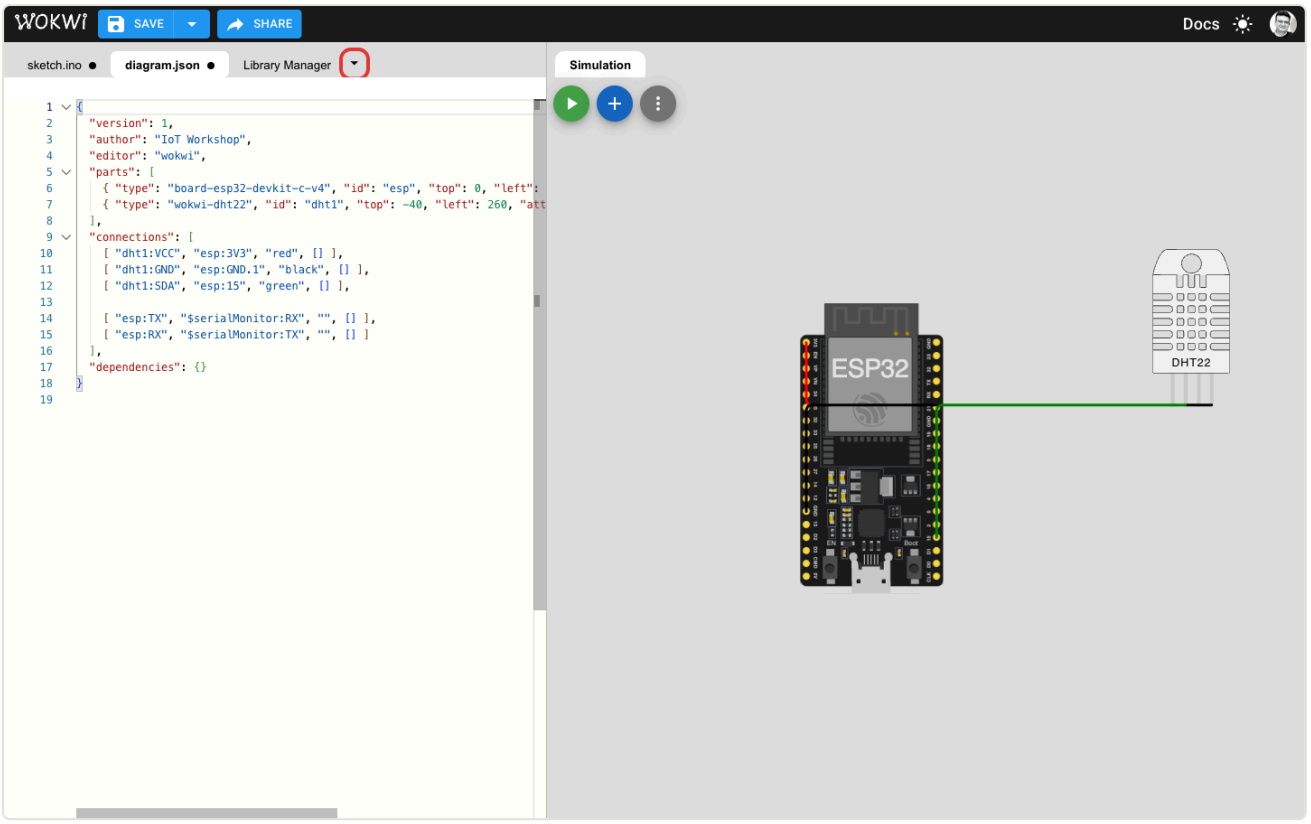
Copy `diagram.json` into Wokwi's **diagram.json** tab. You should see the DHT22, relay module, buzzer, and the "simulated appliance" LED all appear, wired to the ESP32.



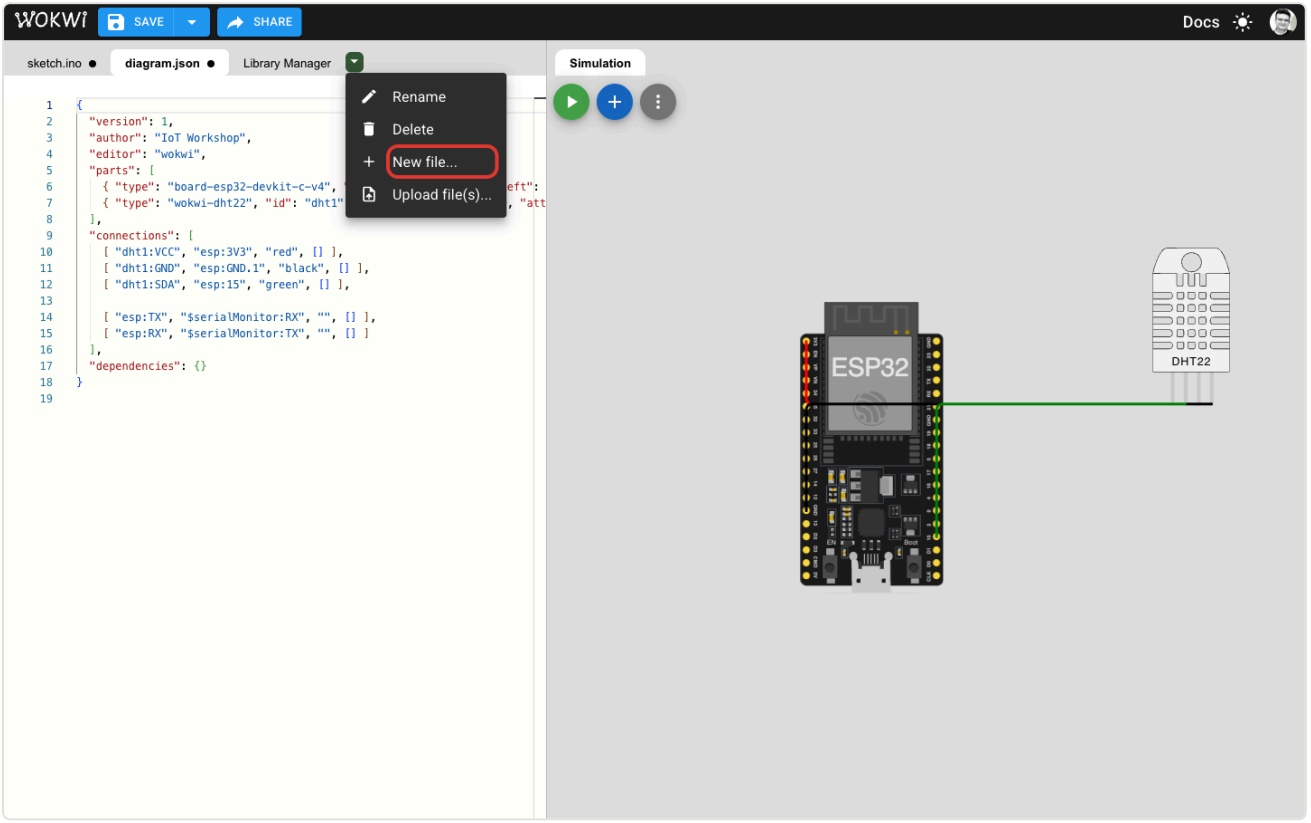
4. Add the library list

This sketch only needs one library, `DHTesp`.

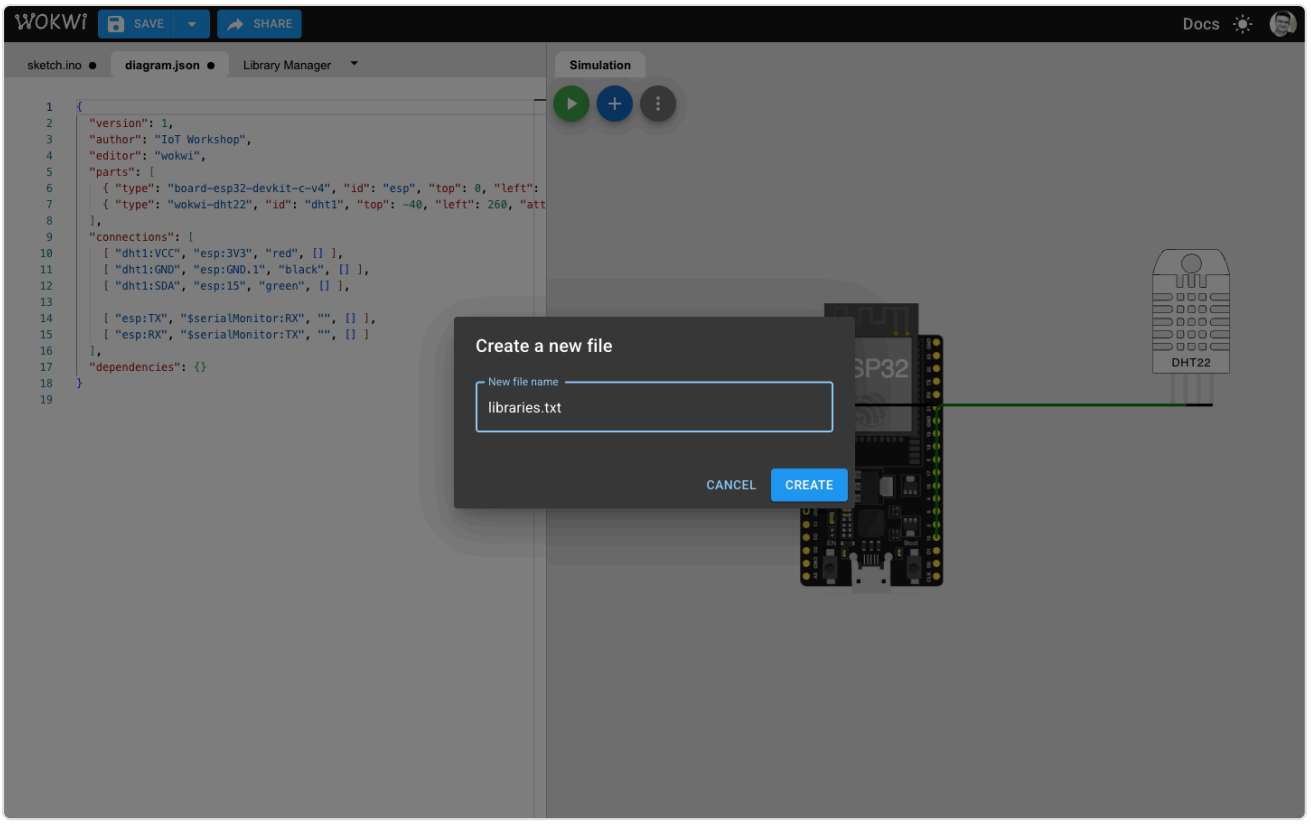
1. Click the small ▾ next to the **Library Manager** tab.



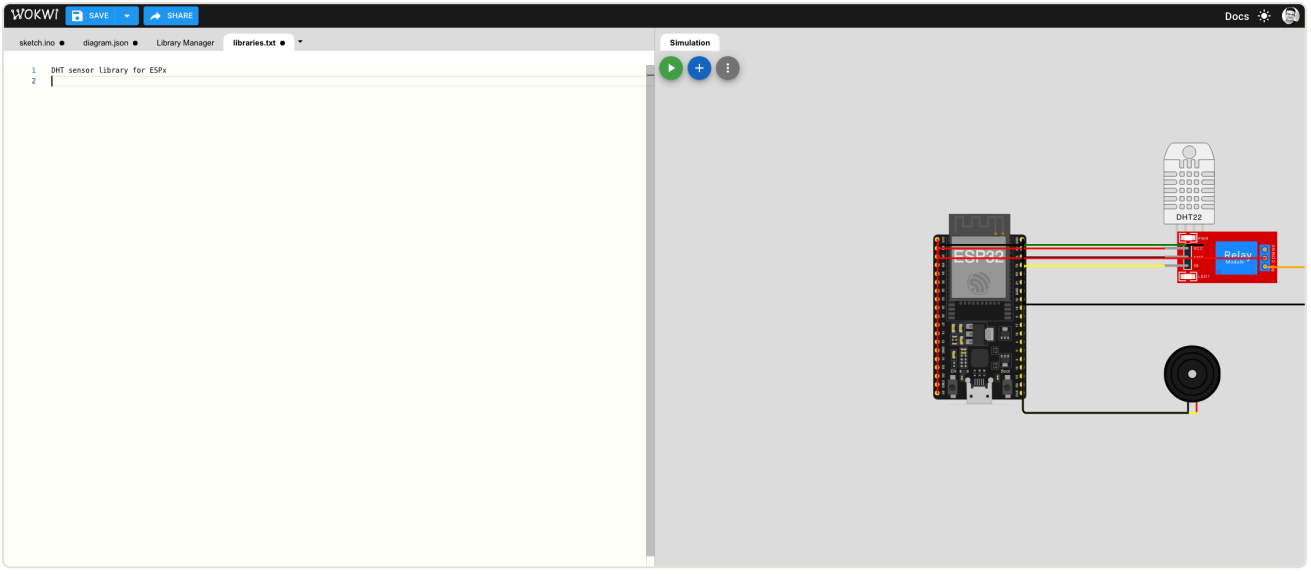
2. Choose **New file...**.



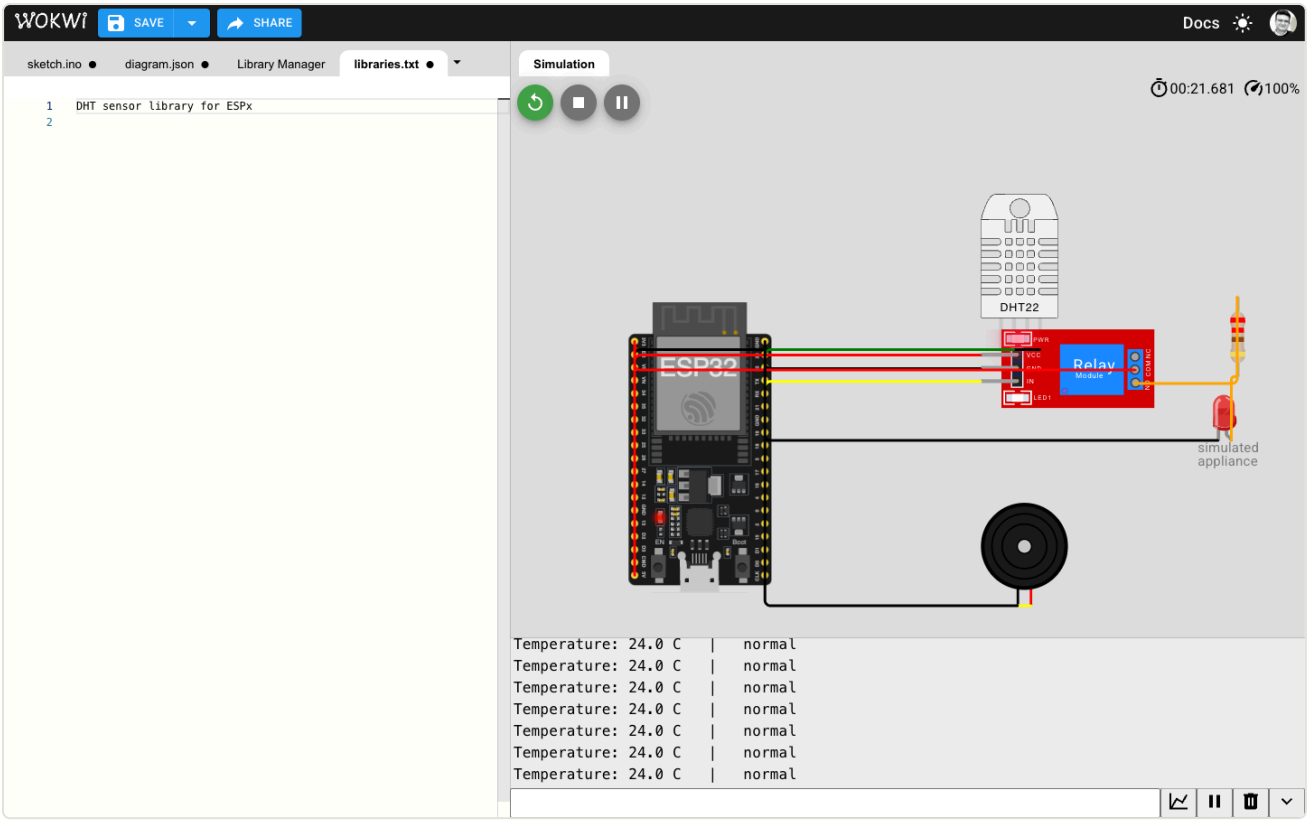
3. Type the name exactly `libraries.txt` and click **CREATE** (or press Enter).



4. Copy the contents of `libraries.txt` from this folder and paste them into the new **libraries.txt** tab.



5. Press Play



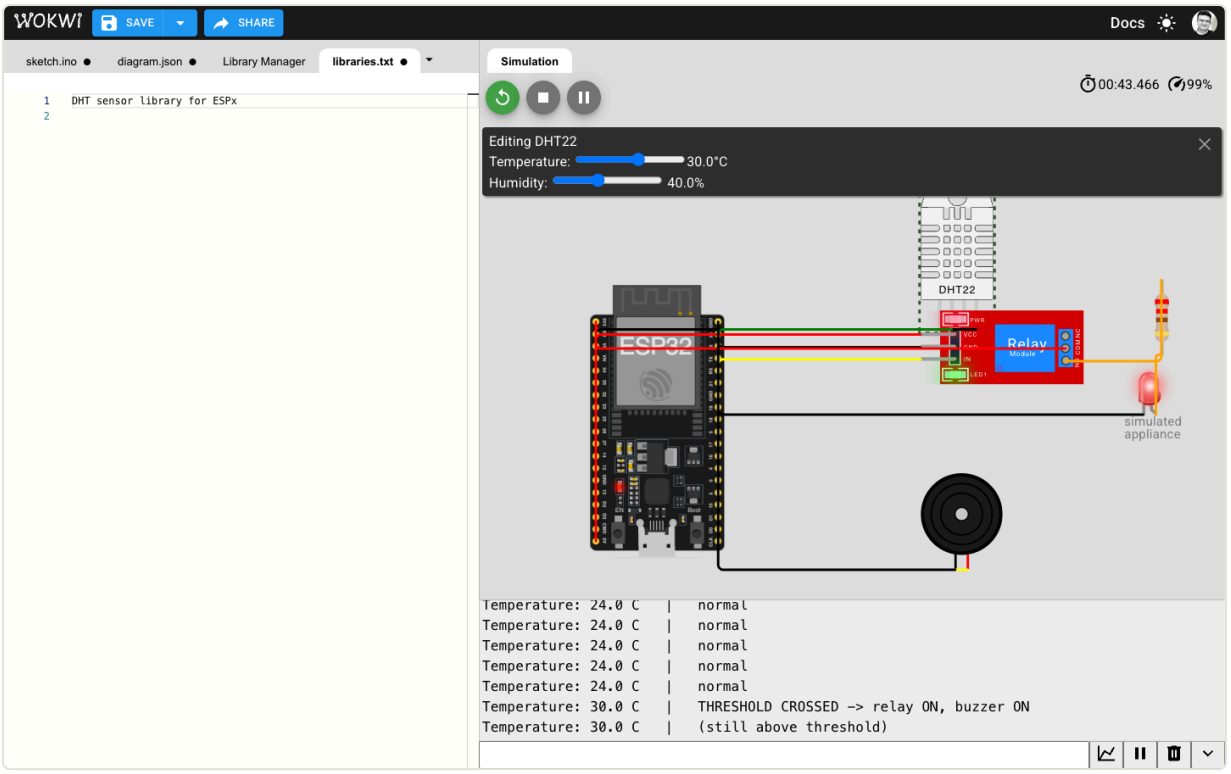
You should see temperature readings every 2 seconds, all reporting “normal” (the default simulated temperature, 24°C, is below the 28°C threshold in the code).

What you should see

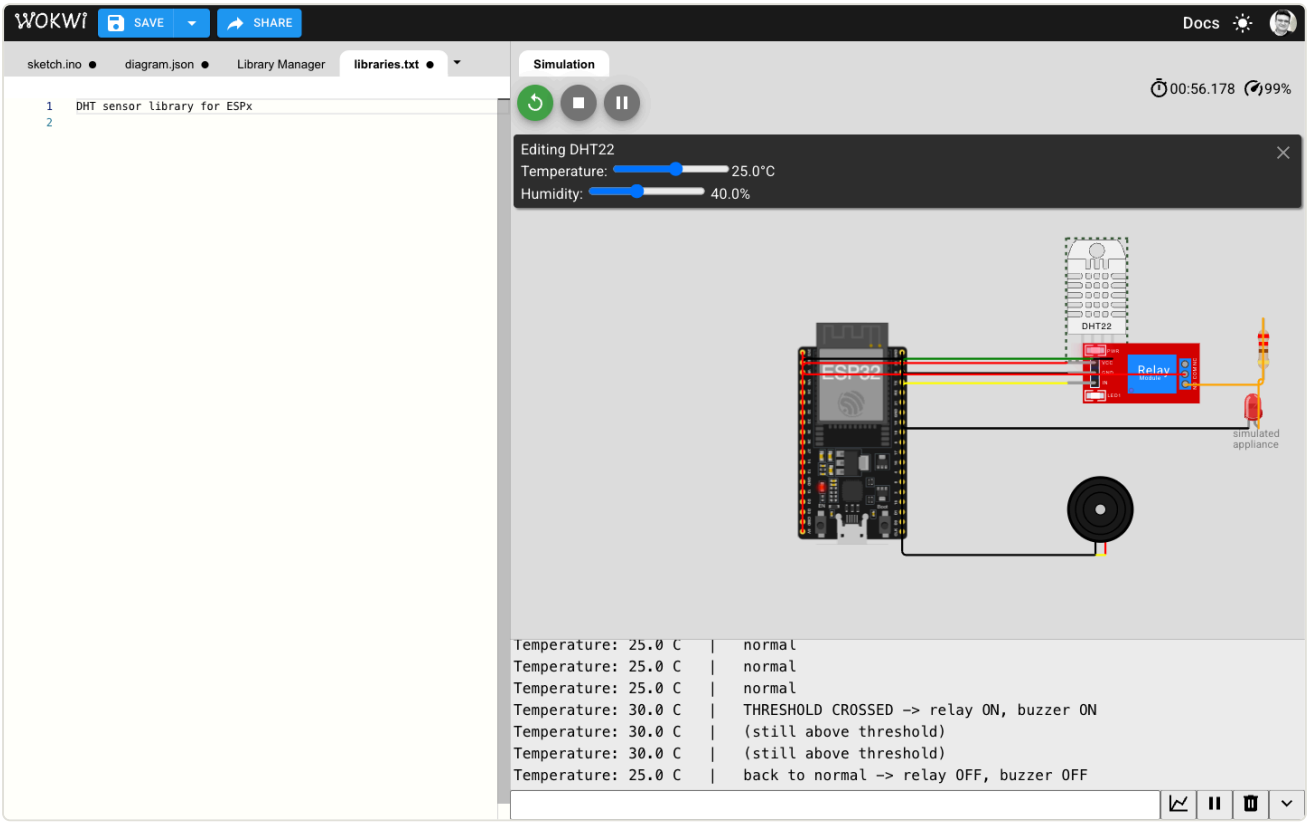
```
Lab 5 started.
Rule: if temperature > 28.00 C, switch on the relay and buzzer.
Click the DHT22 sensor and raise the temperature slider to test it.
Temperature: 24.0 C   |   normal
Temperature: 24.0 C   |   normal
```

Try it

1. While the simulation runs, click the DHT22 sensor and drag the temperature slider above 28°C.
2. Watch three things happen at once, within about 2 seconds: - The Serial Monitor prints `THRESHOLD CROSSED -> relay ON, buzzer ON`. - The relay module’s indicator visibly switches (and you may hear a “click” sound in some Wokwi themes). - The red LED (the “simulated appliance”) turns on.



3. Drag the temperature back down below 28°C and watch everything switch off again, with the “back to normal” message.



Understand the code

- `TEMPERATURE_THRESHOLD_C` is the whole rule, in one line. Real rules engines (in ThingSpeak, AWS IoT, Node-RED, etc.) let you build the exact same kind of if-then logic through a web UI instead of code — but the underlying idea is identical to what you just wrote.
- The relay's `IN` pin is a normal digital output from the ESP32's point of view — `digitalWrite(RELAY_PIN, HIGH)` is exactly the same instruction as turning on an LED. The relay just happens to be strong enough to switch a much bigger circuit (in real life, up to mains voltage) safely.
- `alarmIsOn` is used only so the code doesn't spam "THRESHOLD CROSSED" every single loop — it prints that message once, on the moment of change, which is closer to how a real alerting system behaves.
- Nothing in this code touches Wi-Fi. That's the point: this decision happens entirely on the device — it would keep working even with no internet connection at all.

Troubleshooting

- **`Build failed! ... DHTesp.h: No such file or directory`.** You skipped step 4, or `libraries.txt` doesn't contain exactly `DHT sensor library for ESPx`.
- **Relay/buzzer never turn on.** Make sure you're dragging the DHT22's temperature slider *above* 28.0°C, not just close to it — the rule uses strictly-greater-than (`>`), so exactly 28.0°C stays "normal."
- **Nothing seems to update after crossing the threshold.** Give it up to 2 seconds — the DHT22 can only be read that often, same as Lab 2.

Challenge (if you finish early)

- Add a second rule for humidity (e.g. sound the buzzer, but not the relay, if humidity is above 70%).
- Combine this with Lab 3: publish an MQTT message only at the *moment* the threshold is crossed, instead of a reading every few seconds — this is closer to how real alerting systems avoid flooding a network with repetitive data.